

Computer Architecture Seminar Series for OpenQSE

9 March 2026

Josh Moles
Jiri Schindler, PhD
Spencer Churchill
Guen Prawiroatmodjo, PhD



Josh Moles

josh@ionq.co

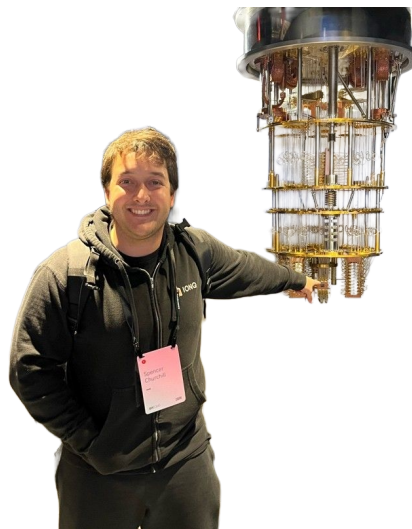
Advancing quantum computing by getting CPUs, GPUs, and QPUs to solve problems easily – and happily – together.



Jiri Schindler, PhD

jiri@ionq.co

Quantum OS architect. Designing the software that brings quantum computers into enterprise data centers – and keeps them running.



Spencer Churchill

churchill@ionq.co

SDK engineer building the bridges between IonQ and the quantum ecosystem – QRMI & QDMI plugins, plus the IonQ integration for the Nvidia + Braket QC-AFQMC demo.



Guen Prawiroatmodjo, PhD

guen.p@ionq.co

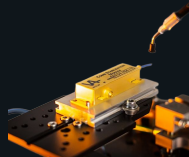
Sr. Engineering Manager leading the Developer Tools team at IonQ, formerly Open Source lead at Riverlane, Software Engineer at MotherDuck and Quantum Engineer at Microsoft Quantum and Rigetti.

01

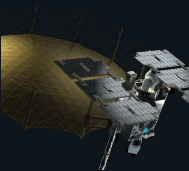
IonQ Overview

Josh

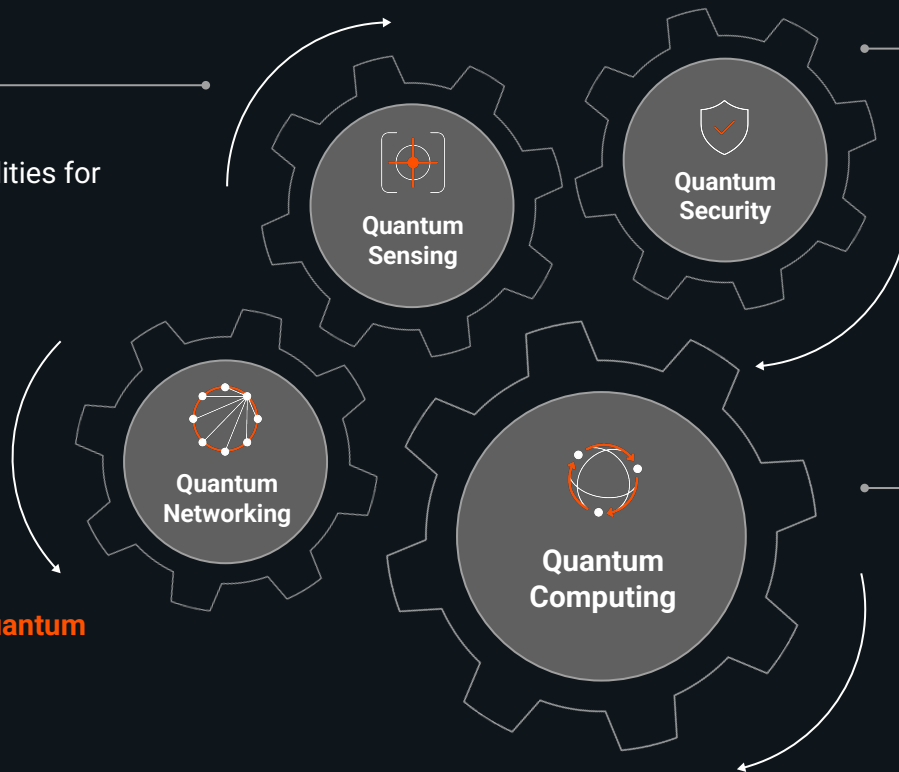
Full Solutions + Leadership in Every Product Family



Leveraging precise measurement capabilities for **ultra-high precision and accuracy**



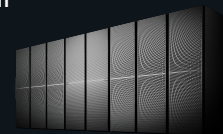
Building the **global quantum internet** for secure communications



Establishing real-world **quantum security with deployable PQC and QKD solutions**



Most powerful systems for **commercial deployment**

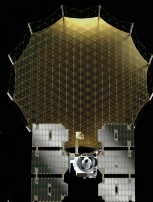
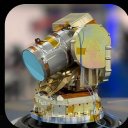


The Only Company Capable of Delivering the **Full Promise of Quantum** Across Compute, Networking, Sensing & Security



Capella & Skyloom

Space and ground infrastructure
enable secure quantum
communication across the globe

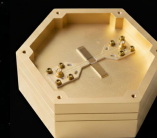


Vector Atomic

Quantum sensing synchronizes systems
perfectly and collects ultra-precise data
for communication, data linking, and
coordination

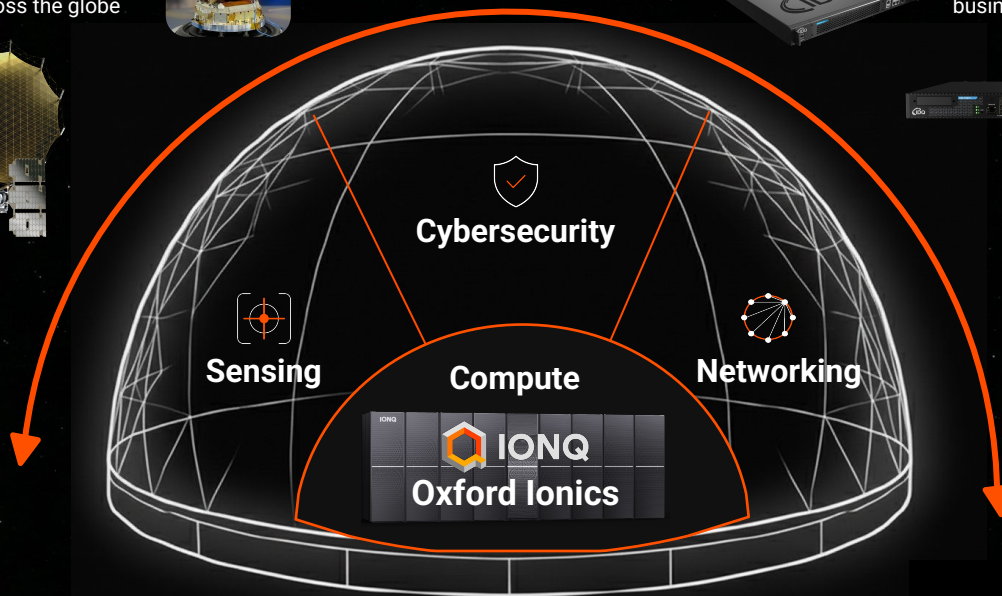
ID Quantique

QKD, PQC and quantum cybersecurity products
protect all data and communications across
businesses and global networks



Lightsynq

Quantum Interconnects enable
distributed computing and secure
communication networks

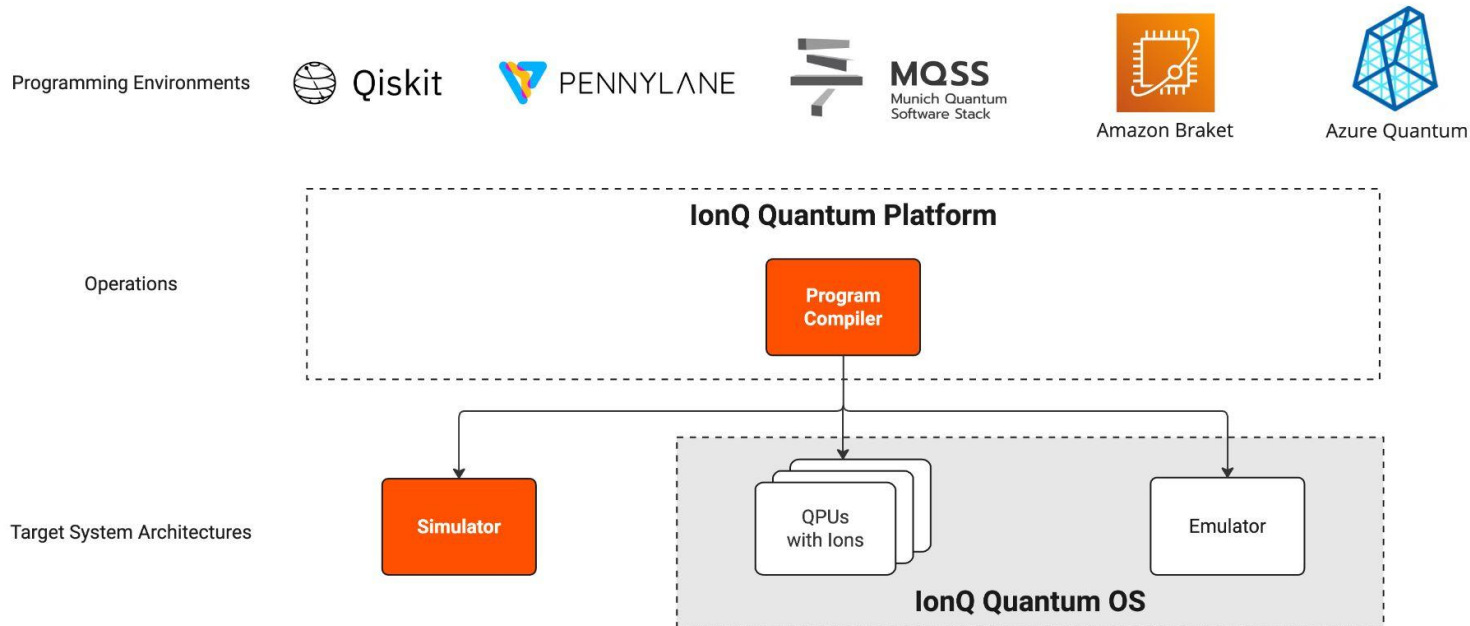


02

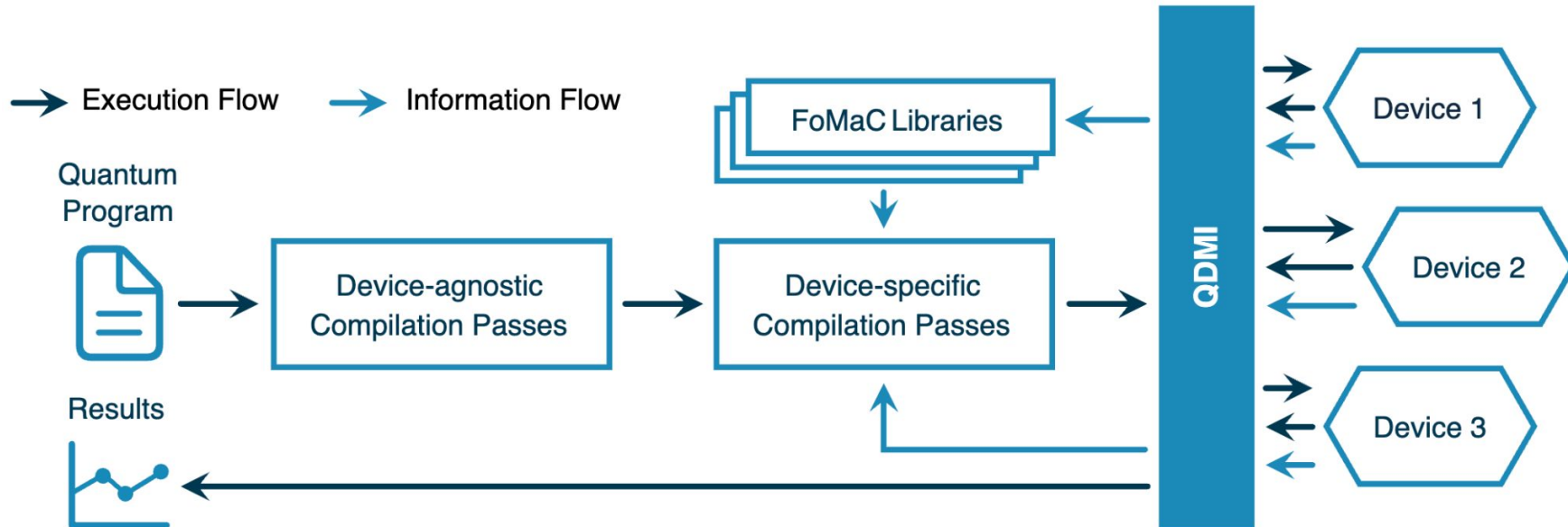
Lifecycle of a Job

Spencer

IonQ Software Layers



QDMI Overview



Source: <https://munich-quantum-software-stack.github.io/MQSS-QDMI-Devices-Suite/index.html>

Mapping QDMI Device Interface to IonQ Constructs

QDMI Device Interface (device.h)

IonQ Component API (api.ionq.co)

Device Queries

QDMI_device_session_query_device_property,
QDMI_device_session_query_site_property,
QDMI_device_session_query_operation_property

GET /v0.4/characterizations/backends/.../current

Job Lifecycle

QDMI_device_session_create_device_job
QDMI_device_job_set_parameter
QDMI_device_job_submit

PUT /v0.4/jobs

QDMI_device_job_check,
QDMI_device_job_query_property

GET /v0.4/jobs

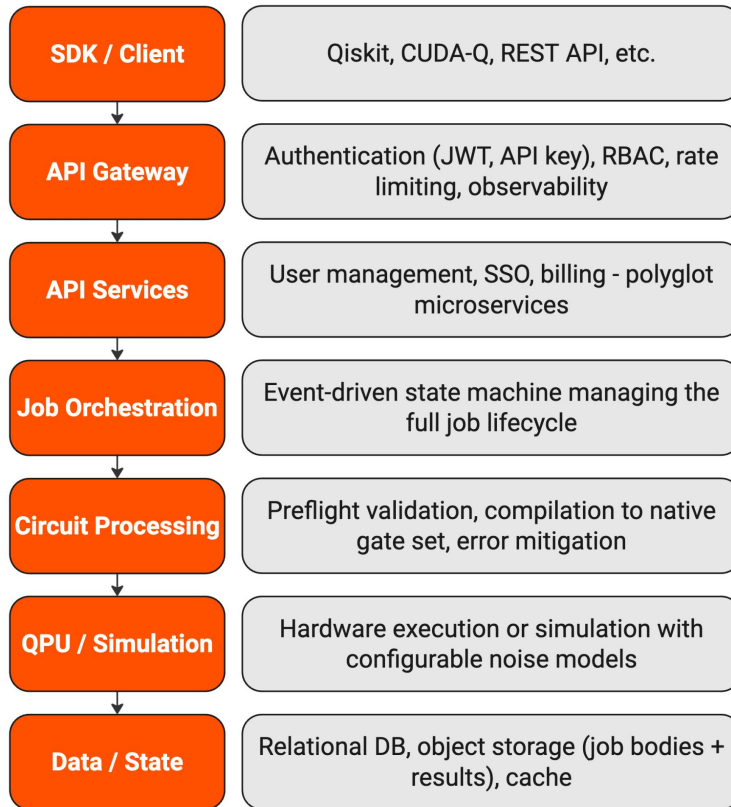
QDMI_device_job_cancel

PUT /v0.4/jobs/.../status/cancel

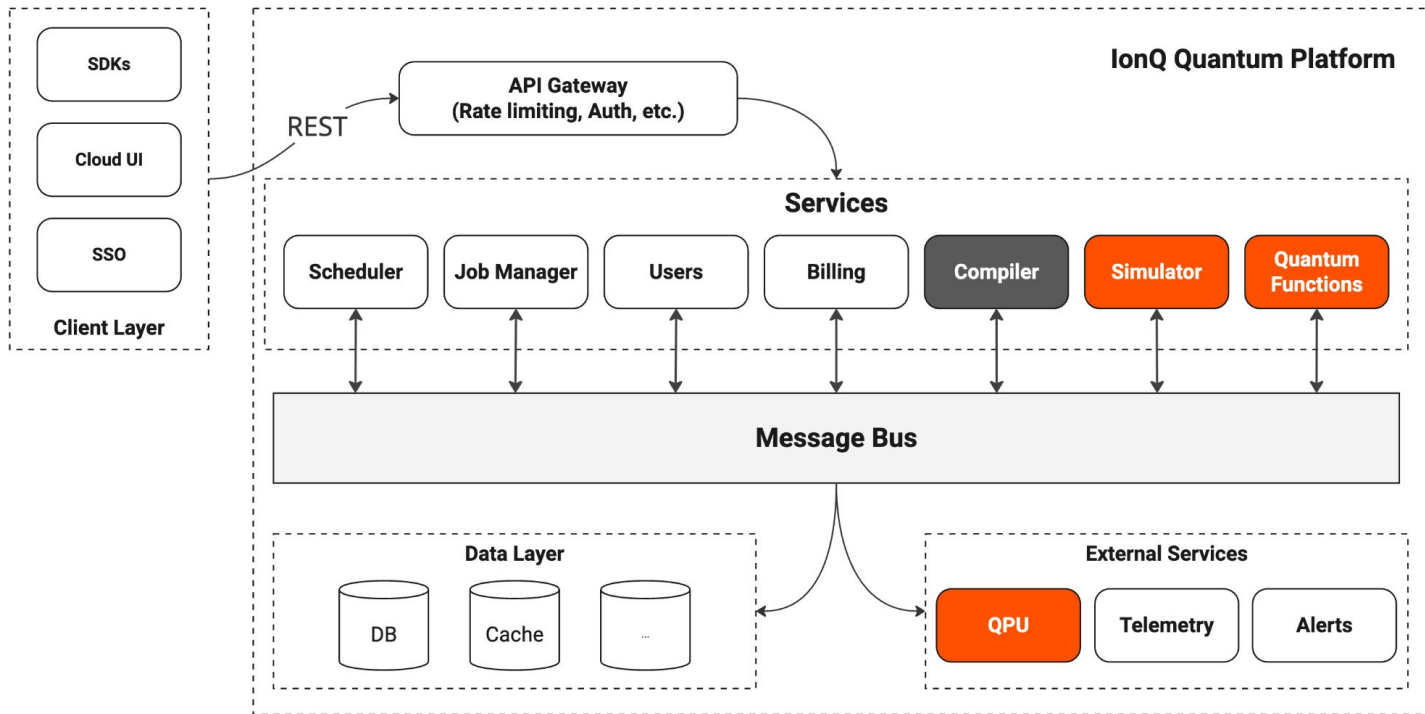
QDMI_device_job_get_results
QDMI_device_job_free

GET /v0.4/jobs/.../results
DELETE /v0.4/jobs/...

IonQ Software Layers



Quantum Platform Architecture with Message Bus



Strengths, Weaknesses, and Surprises

Strengths

Event-driven pluggability

New services subscribe to existing topics; no changes to existing code

End-to-end distributed tracing

OpenTelemetry context flows across various languages & services via pub/sub message attributes

Queue-depth autoscaling

Service replicas scale based on message delivery backpressure

Weaknesses

No shared schema language

Three implementations deserialize pub/sub JSON messages

Inefficient critical path

The job lifecycle runs through scripting languages with runtime inefficiencies

Surprises

High message bus latency

Asynchronous message handling with pub/sub adds overhead

Number of Hops

One job execution hits 8+ pub/sub hops

Solutions

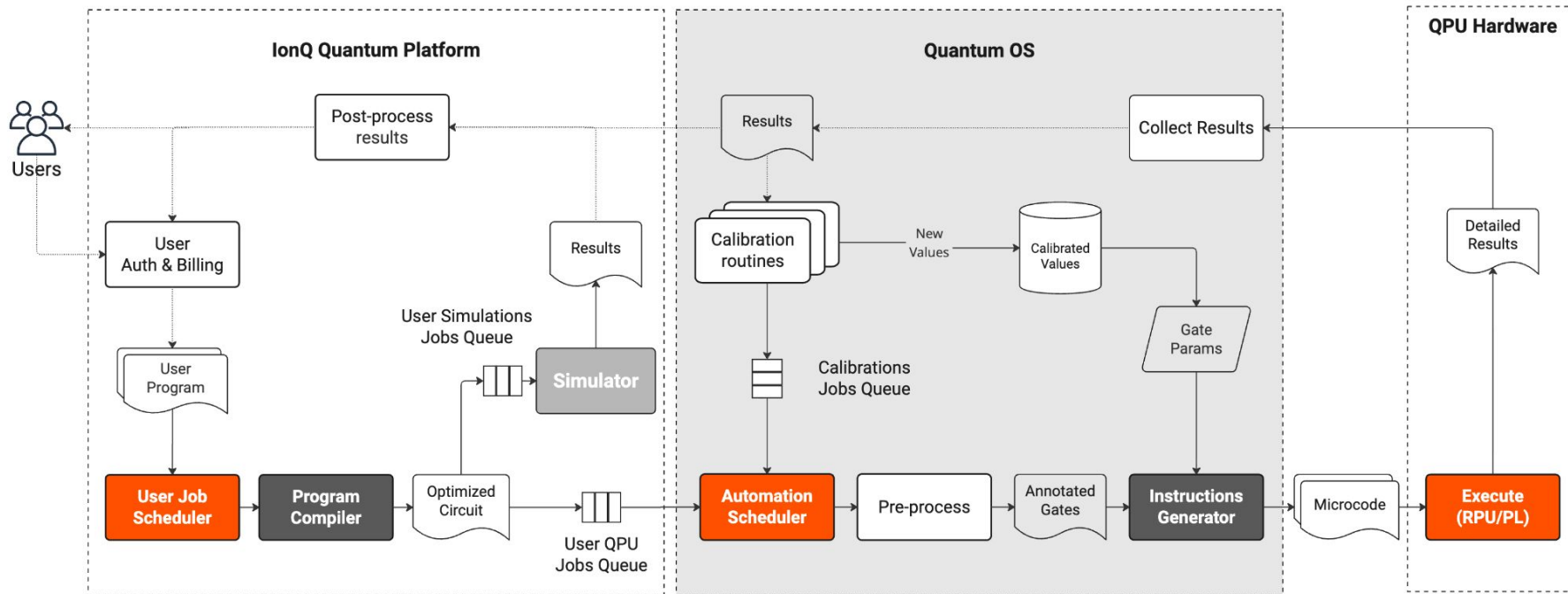
- Rewrite core components in strongly-typed concurrent languages
- Adopt singular method for data exchanges ([Protocol Buffers](#))
- Use remote procedure calls framework ([gRPC](#))

03

Software Stack and Throughput

Jiri

Loosely-coupled Services and Functions



Cooperative Distributed Scheduling

Multi-level job scheduler decouples software and hardware layers

Quantum Platform

Long-term scheduler apportions QPUs to each tenant based on their SLAs

Proportional fair-share scheduling

Reservations for exclusive access

Quantum OS

Fine-grained per-QPU job scheduler

Interleaves user jobs and calibrations

Determines when a QPU is available to run user-submitted jobs

De-queues multiple jobs at the same time for higher execution efficiency

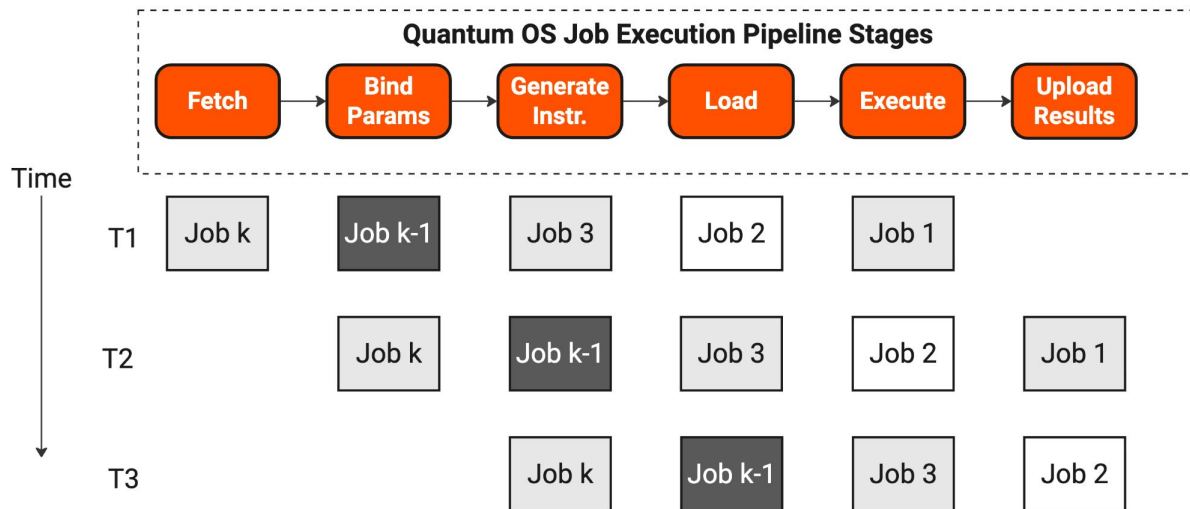
QPU Hardware

Real-time scheduler with ns precision

Quantum OS determines the order of circuit executions on QPU hardware

Increasing per-QPU Throughput with Pipelining

Pipelined execution enables higher QPU utilization



Increasing per-QPU Throughput with Pipelining

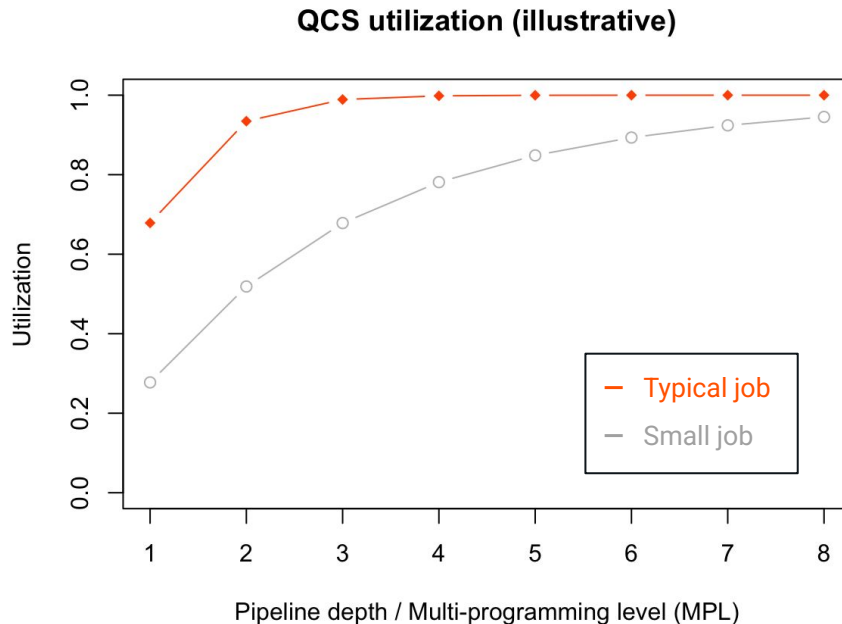
Pipelined execution enables higher QPU utilization

Concurrent execution of multiple user jobs

- pulling several user jobs at once
- still single-threaded QPU execution

Job size affects utilization

- small jobs with fewer gates incur set-up overhead
- large jobs amortize per-job set-up cost



Mechanisms for Efficient Job Execution

- **Proportional fair-share scheduler**
 - Ensures proper prioritization of customer jobs, reservations and prevents starvation
 - Well understood in computer systems literature
- **QPU performance model***
 - Estimates the execution time of an optimized circuit expressed in [native gates](#)
 - Circuit complexity, gate speeds etc. affect total execution time
 - QPU **capacity** is subject to system **availability** & performance
- **Telemetry for accurate accounting**
 - Measures actual execution time of each job
 - Updates estimators & QPU availability based on real-time data
 - Allows long-term data analysis to refine execution predictions

* [Runtime cost estimates](#) available via IonQ Quantum Platform APIs

04

Final Thoughts

Jiri

The Power of Kubernetes

Orchestrating reproducible deployments of versioned software components

- **Production Deployments**
 - SW component upgrades with controlled roll-back
 - Verifiable Software Bill Materials manifests
 - Automated component monitoring and restarts on failure
 - CPU, memory & storage resource management
- **Reproducible Dev and Test Environments**
 - Replication of complete environments for testing and debugging
 - Integration of QPU simulators for full system **emulation**

Interface Definition Languages

Open standards expression of internal proprietary constructs

- **Protocol Buffers**
 - Defines strongly-typed messages for communications between system components
 - Automatic code generation of target language bindings
- **gRPC - remote procedure calls with Protocol Buffers as payloads**
 - Defines contracts and APIs between SW layers and services
 - Automated client & server code generation for concurrent, strongly-typed languages
 - Interface specification **validation** via CI/CD

Leveraging Prior Systems Research

Attribute-based storage management as an analogue

In 2000, storage systems dominated capital & operational costs (Capital & Operational Expenses)

J. Wilkes, [Traveling to Rome: a retrospective on the journey](#). [ACM SIGOPS Operating Systems Review, Volume 43, Issue 1](#), 1-Jan-2009

M. Mesnier, J. Akers, [Differentiated storage services](#). [ACM SIGOPS Operating Systems Review, Volume 45, Issue 1](#), 18-Feb-2011

S. Uttamchandani, K. Voruganti, S. Srinivasan, J. Palmer, and D. Pease. [Polus: Growing Storage QoS Management Beyond a "4-Year Old Kid"](#). In Proc. of the 3rd USENIX Conference on File and Storage Technologies (FAST 04), March 2004. The USENIX Association.

Abstract (Rome retrospective)

Starting in 1994/5, the Storage Systems Program at HP Labs embarked on a decade-long journey to automate the management of enterprise storage systems by means of a technique we initially called attribute-managed storage. The key idea was to provide declarative specifications of workloads and their needs, and of storage devices and their capabilities, and to automate the mapping of one to the other....